Comment configurer un reverse proxy HAProxy sur Debian 12 ?

Charge, Configuration, Haproxy, Linux, Load Balancing

Sommaire [-]

- I. Présentation
- II. Prérequis
- III. Installation de HAProxy sur Debian 12
- IV. Configuration de base d'HAProxy
- A. Configurer le frontend
- B. Configurer le backend
- C. Appliquer la configuration HAProxy
- V. Tester le reverse proxy
- VI. Activer la page des statistiques de HAProxy
- VII. Bonus : créer une ACL sur le frontend HAProxy
- VIII. Conclusion

I. Présentation

Dans ce tutoriel, nous allons apprendre à installer et configurer HAProxy sur un serveur Debian 12, dans le but de déployer un reverse proxy. HAProxy est une application libre et open source utilisée principalement en tant que reverse proxy pour équilibrer la charge et distribuer le trafic réseau entre plusieurs serveurs. Il est fréquemment utilisé pour améliorer la performance, la fiabilité et la sécurité des applications web.
HAProxy est un vaste sujet sur lequel nous pourrions écrire des pages et des pages. Il s'agit d'une introduction pour vous permettre de faire vos premiers pas avec cet excellent reverse **proxy.** Pour approfondir le sujet, vous pouvez consulter la documentation officielle :

- **Documentation officielle de HAProxy**

HAProxy peut s'installer sur **Linux** en tant que paquet autonome et dont la configuration sera effectuée via un fichier de configuration. Sinon, il est aussi possible de le déployer sur un firewall, notamment sur **OpnSense et PfSense**, comme nous l'avions dans un précédent tutoriel :

- **PfSense : reverse proxy HTTPS avec HAProxy et ACME (Let's Encrypt)**

Version originale de l'article : 12 juin 2012.

Prérequis

Pour suivre ce tutoriel, vous devez disposer de **4 machines** afin d'avoir un serveur HAProxy, deux serveurs web et une machine cliente depuis laquelle effectuer les tests.

Pour ma part, les machines suivantes seront utilisées :

- **srv-web-1** avec l'adresse IP "**10.10.1.11**"
- **srv-web-2** avec l'adresse IP "**10.10.1.12**"
- **srv-haproxy** avec l'adresse IP "**10.10.1.13**"

- **srv-wac**, une machine complémentaire avec un navigateur pour accéder au site web

Vous devez avoir un accès administrateur aux serveurs web et au serveur HAProxy. De plus, vous devez préparer le site web sur les serveurs web, ce qui implique de synchroniser les fichiers du site entre les serveurs pour un usage en production.

Pour ma part, il s'agit d'une simple page HTML nommée "**index.html**" avec du contenu statique, stockée sur chaque serveur web. Cette page indique le nom du serveur web, ce qui permettra de voir l'équilibrage effectué par HAProxy.

Si vous avez besoin d'aide pour mettre en place le serveur web Apache2, suivez le tutoriel ci-dessous. Vous pouvez aussi installer un autre service web, comme **Nginx, IIS ou Caddy.**

- **Installer le serveur web Apache2 sur Debian**

Mais, en soi, si vous souhaitez suivre pas à pas ce tutoriel, il suffit d'**installer le paquet Apache2** et de créer une page statique.

```
sudo apt update
sudo apt install apache2
```

×

Désormais, passons à **l'installation et à la configuration de HAProxy**.

Installation de HAProxy sur Debian 12

La première étape pour **mettre en place HAProxy en tant que reverse proxy** est de l'installer sur votre machine **Debian 12**. Debian dispose de HAProxy dans ses dépôts officiels, ce qui évite des étapes intermédiaires.

Avant d'installer HAProxy, il est recommandé de mettre à jour les paquets existants. Ouvrez le terminal et exécutez la commande suivante :

```
sudo apt update && sudo apt upgrade
```

Une fois que c'est fait, installez HAProxy en utilisant la commande suivante :

```
sudo apt install haproxy -y
```

L'installation étant réalisée, vous pouvez vérifier la version installée en saisissant cette commande :

```
haproxy -v
```

La version d'**HAProxy** installée sur la machine Debian sera affichée dans la console. Dans les dépôts actuels de Debian 12, c'est **une version de la branche 2.6** qui est distribuée.

```
haproxy -v
HAProxy version 2.6.12-1+deb12u1 2023/12/16 - https://haproxy.org/
Status: long-term supported branch - will stop receiving fixes around Q2 2027.
```

Cette version LTS sera prise en charge par l'équipe de HAProxy jusqu'au **second trimestre 2027**. Pour passer sur une version LTS plus récente, il est nécessaire de modifier les sources de la machine locale.

Branch	Release date	End of life	Latest version	Changelog
3.1-dev	~2024-Q4	2025-Q4 (dev » stable)	3.1-dev6	2024/08/21
3.0	~2024-05-29	**2029-Q2 (LTS)**	3.0.3	2024/07/11
2.9	2023-12-05	2025-Q1 (stable)	2.9.9	2024/06/14
2.8	2023-05-31	**2028-Q2 (LTS)**	2.8.10	2024/06/14
2.6	2022-05-31	**2027-Q2 (LTS)**	2.6.18	2024/06/18
2.4	2021-05-14	**2026-Q2 (critical fixes only)**	2.4.27	2024/06/18
2.2	2020-07-07	**2025-Q2 (critical fixes only)**	2.2.33	2024/04/05

Source : haproxy.org

Maintenant que HAProxy est installé sur votre système, passons à sa configuration pour le mettre en place en tant que reverse proxy.

Configuration de base d'HAProxy

Le fichier de configuration principal de HAProxy est le suivant :

```
/etc/haproxy/haproxy.cfg
```

Vous devez éditer ce fichier avec un éditeur tel que nano ou vim afin de lui apporter des notifications. Par habitude, j'utilise l'éditeur nano.

```
sudo nano /etc/haproxy/haproxy.cfg
```

À la suite des lignes déjà présentes dans ce fichier, vous devez déclarer la configuration de votre **frontend** et de votre **backend**.

```
flo@srv-haproxy: ~
  GNU nano 7.2                                    /etc/haproxy/haproxy.cfg *
        ssl-default-bind-ciphersuites TLS_AES_128_GCM_SHA256:TLS_AES_256_GCM_SHA384:TLS_
        ssl-default-bind-options ssl-min-ver TLSv1.2 no-tls-tickets

defaults
        log     global
        mode    http
        option  httplog
        option  dontlognull
        timeout connect 5000
        timeout client  50000
        timeout server  50000
        errorfile 400 /etc/haproxy/errors/400.http
        errorfile 403 /etc/haproxy/errors/403.http
        errorfile 408 /etc/haproxy/errors/408.http
        errorfile 500 /etc/haproxy/errors/500.http
        errorfile 502 /etc/haproxy/errors/502.http
        errorfile 503 /etc/haproxy/errors/503.http
        errorfile 504 /etc/haproxy/errors/504.http

# Configuration personnalisée

^G Aide        ^O Écrire      ^W Chercher    ^K Couper      ^T Exécuter    ^C Empla
^X Quitter     ^R Lire fich.  ^\ Remplacer   ^U Coller      ^J Justifier   ^/ Aller
```

Configurer le frontend

Cette section définit comment HAProxy recevra les connexions entrantes, en provenance des clients qui souhaitent se connecter à notre site web. La configuration de HAProxy peut contenir plusieurs frontends.

Ici, nous allons déclarer un frontend nommé "**front_webservers**" accessible sur le **port 80**. Il sera associé au backend nommé "**backend_webservers**". L'option "**forwardfor**" sera déclarée pour que l'en-tête "**X-Forwarded-For**" soit ajoutée aux requêtes HTTP afin que le serveur web ait connaissance de l'adresse IP d'origine du client (sinon, il verra toujours l'adresse IP du HAProxy).
Ce qui donne :

```
frontend front_webservers
        bind *:80
        default_backend backend_webservers
        option forwardfor
```

Vous devez comprendre que tout le trafic à destination du port 80 de HAProxy sera redirigé vers les serveurs web.

> **Remarque** : pour aller plus loin et rediriger les flux en fonction de la requête, notamment selon le nom de domaine que souhaite visiter l'utilisateur, il est nécessaire de déclarer des ACL dans le frontend (via l'instruction "**acl**"). Ici, il s'agit d'une configuration simplifiée basée sur HTTP et les adresses IP.

Configurer le backend

Cette section définit les serveurs vers lesquels HAProxy redirigera le trafic correspondant aux flux à destination du frontend "**front_webservers**", puisqu'il est associé au backend que nous configurer.

HAProxy va distribuer le trafic entre les deux serveurs web nommés "**srv-web-1**" et "**srv-web-2**". La méthode d'équilibrage de charge "**roundrobin**" sera utilisée, ce qui signifie que HAProxy alternera entre chaque serveur web. Dans l'exemple ci-dessous, vous devez penser à adapter les adresses IP des serveurs backends, et éventuellement, les noms.
Ce qui donne :

```
backend backend_webservers
        balance roundrobin
        server  srv-web-1 10.10.10.11:80 check
        server  srv-web-2 10.10.10.12:80 check
```

Si vous utilisez nano, après avoir effectué ces modifications, sauvegardez le fichier en appuyant sur **CTRL + O**, puis **Entrée**, et quittez nano avec **CTRL + X**.

Appliquer la configuration HAProxy

Pour appliquer la nouvelle configuration, redémarrez le service HAProxy avec la commande suivante :

```
sudo systemctl restart haproxy
```

Votre HAProxy est maintenant configuré pour agir comme un reverse proxy, redirigeant le trafic vers vos serveurs backend. Nous allons vérifier que tout fonctionne correctement.

Tester le reverse proxy

Après avoir configuré HAProxy, il est important de vérifier que le service fonctionne correctement et redirige le trafic comme prévu. Tout d'abord, vous pouvez visualiser le statut de HAProxy avec la commande suivante :

```
sudo systemctl status haproxy
```

Cette commande indique aussi l'état récent des serveurs backends, ce qui permet de voir, si du point de vue de HAProxy, ils sont "up" ou "down".

```
# Serveur hors ligne (down)
```

```
Server backend_webservers/srv-web-1 is DOWN, reason: Layer4 connection problem,
# Serveur en ligne (up)
Server backend_webservers/srv-web-1 is UP, reason: Layer4 check passed
```

À partir de la machine cliente, vous pouvez accéder à votre site web via l'adresse IP du serveur HAProxy : **http://10.10.10.13**. Sur l'image ci-dessous, nous pouvons constater qu'à chaque fois que l'on rafraichit la page, nous sommes basculés d'un serveur web à l'autre. Ceci est conforme au mode Round Robin.

Activer la page des statistiques de HAProxy

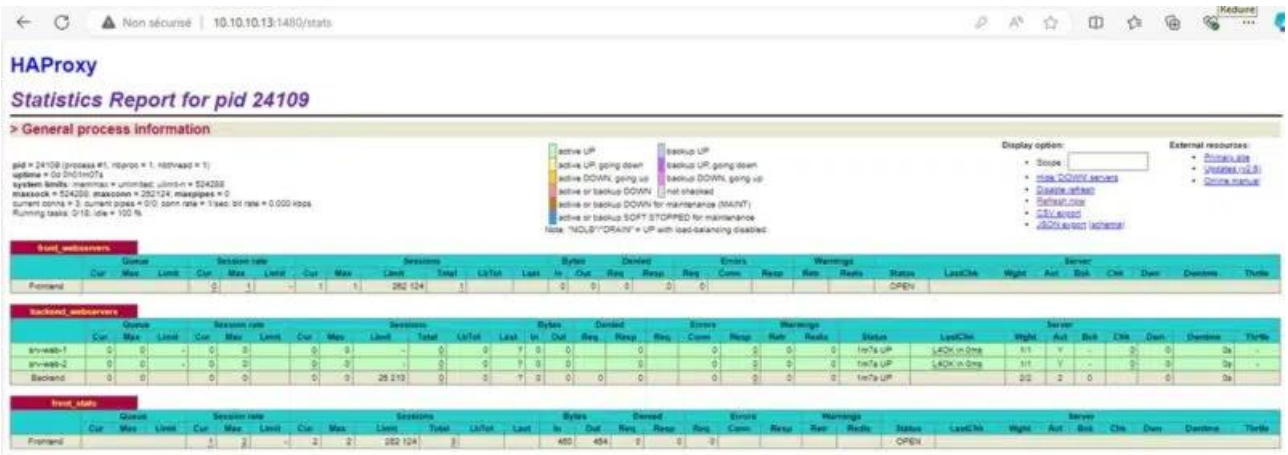
HAProxy embarque une interface prête à l'emploi permettant d'avoir une vue d'ensemble sur les frontends et les serveurs de backends, avec un ensemble de statistiques. De plus, cette page indique l'état de chaque serveur de backend.

Pour activer et configurer cette page de statistiques, vous devez éditer le fichier de configuration "**haproxy.cfg**". Puis, ajoutez ceci à la fin du fichier, à la suite de la configuration précédente, pour déclarer le frontend "**front_stats**" :

```
frontend front_stats
    bind *:1480
    stats enable
    stats uri /stats
    stats hide-version
    stats refresh 10s
    stats realm Haproxy\ Statistics
    stats auth haproxy:P@ssword!
```

L'interface sera accessible sur le port "**1480**" (*bind*) en indiquant l'adresse IP du serveur HAProxy suivi de "**/stats**" (*stats uri*). Les statistiques de cette page seront actualisées **toutes les 10 secondes** (*stats refresh*) et elle sera **protégée par un identifiant et un mot de passe** (*stats auth*). Dans l'exemple ci-dessus, l'utilisateur s'appelle "**haproxy**" et le mot de passe est "**P@ssword!**". Bien entendu, adaptez ces valeurs.

Enregistrez la configuration, redémarrez HAProxy et tentez d'accéder à la page suivante : "**http://10.10.10.13/stats**". Après avoir saisi les identifiants, vous devriez arriver sur cette page :



Nous pouvons constater que les deux serveurs web en backend sont actuellement en ligne. Cette page est très riche en information. Pour en savoir plus, je vous recommande de lire cet article du blog officiel.

VII. Bonus : créer une ACL sur le frontend HAProxy

Avec la configuration actuelle, si un utilisateur se connecte sur l'adresse IP du serveur HAProxy, il sera en mesure d'accéder à nos serveurs web. Néanmoins, en production, nous allons plutôt travailler avec un nom de domaine, qu'il soit interne ou publique.

De ce fait, il est préférable que la requête soit redirigée vers les serveurs backends uniquement si elle correspond à notre nom de domaine. C'est un moyen de protéger les serveurs web et de ne pas autoriser les accès basés sur l'adresse IP. Pour cela, nous devons créer une règle d'ACL dans la configuration du frontend HAProxy.

Pour cet exemple, nous allons uniquement autoriser les requêtes à destination du nom de domaine "**app.it-connect.local**" correspondant à une application fictive. Pour faire le test, vous pouvez modifier le fichier "**hosts**" de votre machine cliente ou créer une entrée dans la zone DNS interne (app.it-connect.local doit correspondre à l'adresse IP de HAProxy).

Éditez le fichier de configuration de HAProxy puis modifiez le frontend "**front_webservers**" de cette façon :

```
frontend front_webservers
        bind *:80
        # default_backend backend_webservers
        # ACL pour app.it-connect.local"
        acl ACL_app.it-connect.local hdr(host) -i app.it-connect.local
        use_backend backend_webservers if ACL_app.it-connect.local
        option forwardfor
```

Nous créons une règle ACL (*Access Control List*) pour définir une condition basée sur l'en-tête HTTP "**Host**" d'une requête. Nous cherchons à vérifier que c'est bien le nom de domaine "**app.it-connect.local**" qui est visité, sans tenir compte de la casse.

Si l'en-tête "**Host**" d'une requête HTTP contient exactement "**app.it-connect.local**", alors l'ACL sera vraie, et dans ce cas, nous redirigeons le flux vers les serveurs du backend nommé "**backend_webservers**". Ce backend ne doit pas être défini comme "**default_backend**" pour ne pas permettre les flux sur l'adresse IP.

Conclusion

Nous avons vu comment installer et configurer HAProxy en tant que reverse proxy sur Debian 12. Grâce à ce tutoriel, vous avez appris à rediriger le trafic entrant vers plusieurs serveurs backend, assurant ainsi une répartition de la charge et une haute disponibilité pour votre site web fictif.

Pour une configuration en production, il sera nécessaire de créer des règles ACL basées sur les noms de domaine pour rediriger vers les serveurs web (backend) uniquement certaines requêtes. De plus, la prise en charge des requêtes HTTPS (TLS) sera indispensable pour la sécurité.

Si vous souhaitez plus de tutoriels sur HAProxy, n'hésitez pas à nous en faire part en postant un commentaire sur cet article.